

```
# julia juliatime35.jl
Time taken: 3.13834810256958 seconds
The factors are BigInt[2, 30993028074067, 2458872160579053728242368539]
# sage sagetime36.sage
Wall time: 0.10870647430419922 seconds
The factors are: 2 * 30993028074067 * 2458872160579053728242368539
```

Figure 1: Julia and SageMath speed comparison

SageMath knocked out Julia, with SageMath enjoying roughly a 29-fold advantage. Moreover, as the size of the number stored in the variable n increases, SageMath's performance advantage becomes even greater.

While Julia is widely recognised as a fast programming language for scientific and numerical computing, SageMath holds a distinct advantage in certain domains—particularly in number theory and integer factorisation. This is because SageMath integrates highly optimised external libraries such as PARI/GP and ECM under the hood, which are renowned for their efficiency in handling large integers. In contrast, Julia's built-in `factor()` function, provided by the package *Primes.jl*, tends to be significantly slower when dealing with large numbers.

Now, let us explore a few examples that showcase the power of PARI/GP and ECM directly, rather than letting them work quietly behind the scenes. To illustrate this, consider the SageMath program 'powersage37.sage', which executes two PARI/GP one-liners and two ECM one-liners, demonstrating their capabilities in action.

```
print(pari("default(realprecision,100); Pi"))
print(pari("50!"))
print(EllipticCurve(GF(101), [randint(0,100),
randint(0,100)]))
print(EllipticCurve(GF(101), [0, 0, 1, -1, 0]).random_
point().order())
```

Now, let us try to understand the one-liners. The first one-liner uses PARI/GP to compute a high-precision value of π , while the second one-liner computes the factorial of a large number using PARI/GP. The third and fourth one-liners perform operations on elliptic curves using ECM, a special type of curve with numerous applications in public-key cryptography (PKC). A full theoretical explanation of elliptic curves is beyond the scope of this series, but these examples clearly demonstrate the power of ECM and its capabilities within SageMath.

Upon executing the program 'powersage37.sage', the output shown in Figure 2 is obtained. As the figure illustrates, the integration of PARI/GP and ECM provides SageMath with

a significant boost in computational power and versatility.

Diffie–Hellman key exchange

Now, it is time for us to go back to a period even before the birth of RSA — a time when the very concept of public-key cryptography did not exist. In the early 1970s, two computer scientists, Whitfield Diffie and Martin Hellman, were grappling with a fascinating problem: How can two parties securely exchange a secret key over an insecure communication channel? Their collaboration led to the groundbreaking Diffie–Hellman (DH) key exchange protocol, introduced in 1976. This was the first practical method that allowed two people to establish a shared secret key without having met or exchanged anything privately beforehand — a truly revolutionary idea for its time.

However, while their discovery laid the foundation for modern cryptography, Diffie and Hellman stopped short of realising that the same mathematical principles could also be used to create a system for direct encryption and digital signatures. In other words, they solved the problem of secure key exchange, but not secure message encryption using public and private keys. It was only later, in 1977, that Rivest, Shamir, and Adleman at MIT took the next leap by inventing the RSA algorithm, which implemented the full idea of public-key cryptography (PKC) — a concept that Diffie and Hellman had unknowingly set the stage for.

Now, consider the SageMath program named 'toydh38.sage', which implements a toy version of the Diffie–Hellman key exchange. In this example, small numbers are used to make the underlying process easy to follow and to help visualise how the Diffie–Hellman key exchange works in principle.

```
p = 23
g = 5
a = 6
A = power_mod(g, a, p)
b = 15
B = power_mod(g, b, p)
S_alice = power_mod(B, a, p)
S_bob = power_mod(A, b, p)
print("Public parameters: p =", p, ", g =", g)
print("Alice sends A =", A)
print("Bob sends B =", B)
print("Alice computes secret:", S_alice)
print("Bob computes secret: ", S_bob)
```

```
# sage powersage37.sage
3.141592653589793238462643383279502884197169399375105820974944592307816406286208998628034825342117068
304140932017133780436126081660647688443776415689605120000000000000
Elliptic Curve defined by y^2 = x^3 + 75*x + 77 over Finite Field of size 101
9
```

Figure 2: PARI/GP and ECM code